

COS214 Practical 6

Christian Khaled u25420314 sexy thing

Keagan Coetzee u25122003

Nathan Spavins u25054245

Jack Reid u25265718

Simon Vogel u25087984

Task 1: Research Workflow Management Systems

1) What is a workflow and what is a workflow management system?

A workflow is a repeating process where tasks happen in a specific order. A workflow management system is the software that actually runs it, sending each task to the right person when it's their turn.

2) What is the difference between a workflow definition and a running instance?

A definition is the template for a process. An instance is one real case worked through from that template, so one definition can start many separate cases.

3) How are activities or work items represented?

Activities are the steps of a process, and they happen in a set order. Systems often draw them with BPMN (Business Process Model and Notation), a standard set of symbols for drawing business processes. It splits the flow into different paths using decision points called gateways, and parallel paths are tracked separately.

4) How are work items assigned and routed?

The system sends each task to whoever should do it, when it needs doing. Gateways decide which path the work takes next.

5) How are approvals, rejection and escalation handled?

Timers, errors and escalations can be attached to a task as events. Each can stop the task or let it carry on, which is how a slow task gets passed to someone higher.

6) How are participants, roles and responsibilities represented?

Process diagrams use swimlanes to show who does what, and some tasks are marked as work done by people. The system's job is to get each task to the right person.

7) How are dependencies, events and failures handled?

The order of the steps sets the dependencies, so a step waits for the one before it. Messages, timers and errors are events the process can catch, so a failure can be raised and handled.

8) How do workflow systems interact with external systems?

There are tasks for calling other systems and for sending or receiving messages. Automated tasks run alongside human tasks in the same process.

9) How is workflow history or audit information recorded?

The engine stores where each live case is up to, and keeps a separate log of what has already happened.

10) How are running workflows monitored and reported on?

The engine keeps records of current and past cases, and monitoring tools use them to show what is happening. Camunda, a workflow engine, has a web tool called Cockpit for this.

11) What makes a workflow engine reusable across different processes?

A process is written as a model and handed to the engine, which runs it. The engine has no process of its own, so it can run any process.

12) Which information belongs to a definition and which to an instance?

The definition holds the structure of the process and its version. The instance holds where that case currently is and what has already happened in it.

13) How do real systems model nested activities or sub-processes?

A process can contain smaller processes, called sub-processes. Each one is tracked as its own part of the running case, and some can be reused from many places.

14) Which decisions are normally configurable instead of hard-coded?

The process itself is loaded in as a model, not written into the engine. Settings like how much history is kept are also changed through configuration.

15) How are changes to definitions handled while work is in progress?

An updated process is added as a new version next to the old one, so several versions can exist side by side.

References

- Camunda (n.d.) *BPMN 2.0 Implementation Reference*.
<https://docs.camunda.org/manual/reference/bpmn20/>
- Camunda (n.d.) *History and Audit Event Log*.
<https://docs.camunda.org/manual/7.19/user-guide/process-engine/history>
- Camunda (n.d.) *Process Engine Concepts*.
<https://docs.camunda.org/manual/7.9/user-guide/process-engine/process-engine-concepts>
- IBM (n.d.) *What is a workflow?* <https://www.ibm.com/topics/workflow>

Task 2: Define the Scenario and System

TIVIDY is a generic workflow management system whose main responsibility is to build and store workflow definitions, and use those definitions to create instances of that definition.

Another responsibility is to work out what work to do next, based on the definition, by looking at what has been finished and what tasks are required next. On top of this it decides who

does each task. It also tracks each tasks progress to ensure only valid tasks are running. It also record the history in order to go back to a previous state or undo what was just done in the system. When something happens it needs to notify all the interested parts of the system involved in case they need to change. It also sends tasks through the right approvers.

The boundaries of this system are that it temporarily doesn't allow for external systems as we are focused on the workflow design, not external additions. There is no database, networking, or GUI, as all things are accessed through the one single interface, and data is kept in the memory

The information that is in the workflow definition is the same information for every run, it stores the workflow name, the stages, tasks, order that they happen, parallel tasks, roles needed, rules, and approvals. The instance rather is a working copy of the definition. Each instance has its own specific data and changing it doesn't change others or the definition, just its own data. It's like an object class (definition), and the actual objects (instances).

The TIVIDY needs to vary and work differently for different organisations. The Important parts include, assigning work, where you could have one organisation that gives tasks via a role, or a different organisation based of who is the least busy. Approving tasks could have some organisations that need 1 approver, whereas others need more. When it comes to the tasks behaviour some organisations might want tasks to be validated, have a deadline, or priority, however others might not want any behaviour on their tasks. Reacting to event will have organisations wanting to react in different ways like notifying or updating. The last important part is the actual process as each organisation is going to have different tasks, stages, etc.

Scenario:

- The organisation is a university that receives many students applying for undergraduate degrees. Without a workflow system this could take a long time to organize the applications and be a complicated process.
- The processes being managed are the students sending in their applications for the degree program, the system starts with the application and ends with the student getting a result
- The participants and roles are the applicant who submits the application, admissions officer checks the documents, payment, and can undo mistakes. The Head of degree who looks at the application, and the head of the faculty who is the higher level of approval over the head of degree.
- The major pieces of work are submitting the application. Have the admissions officer verify everything. Have a head of degree assess application, have head of faculty approve application and lastly send the decision to the applicant.
- The important decisions to be made checking if the documents are valid, who checks the documents, which head of degree reviews it and what they decide, and is it valid to be accepted and meets degree requirements.
- A dependency can be that a review cant be done until a document is valid. And a application will be approved by the head of degree, or the head of the faculty.

The head of the degree will have 5 days to respond or it will escalate to the head of faculty.

- Failures in this system include documents missing, failed payments, no response, incorrect decision made, or an applicant cancels.
- External systems that could be used, are a payment system and email system for the transfer of data from the applicant to the system.
- The expected final outcome of the workflow should be a final decision, accepted, waitlisted, or declined having every action being stored.

Task 3: Activity Diagrams

Diagram 1: Generic Workflow Execution

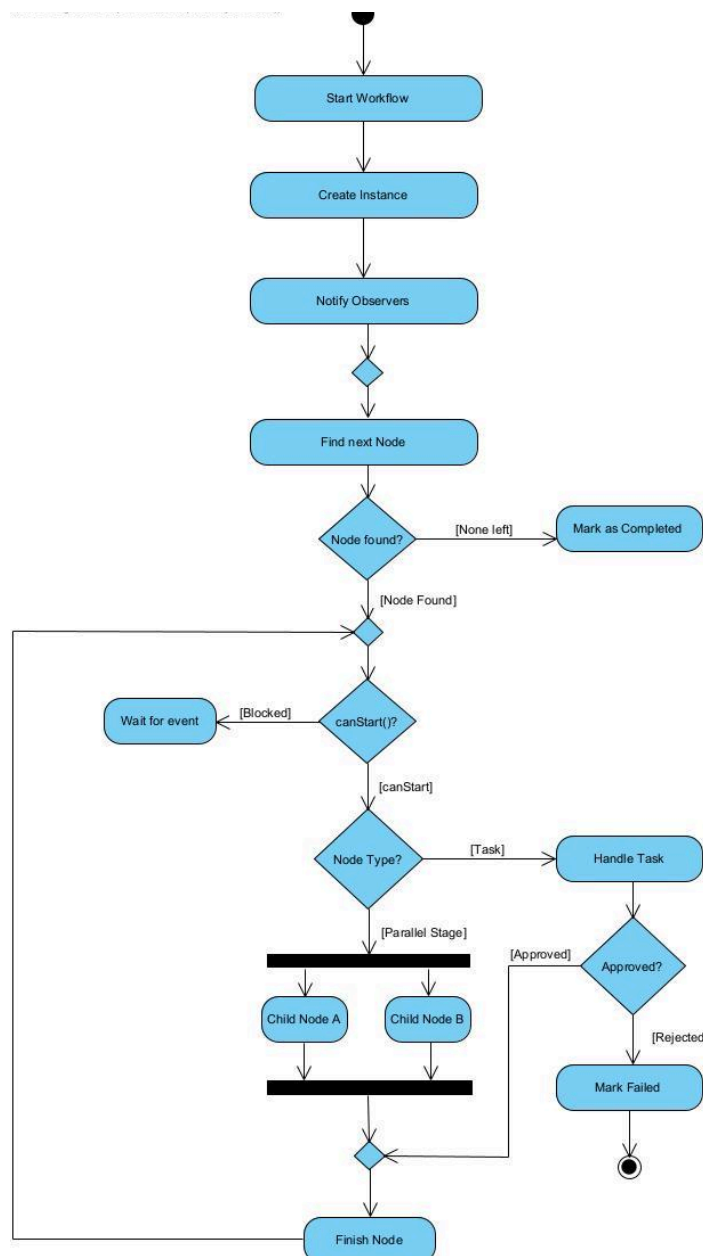


Diagram 2: Generic TIVIDY workflow

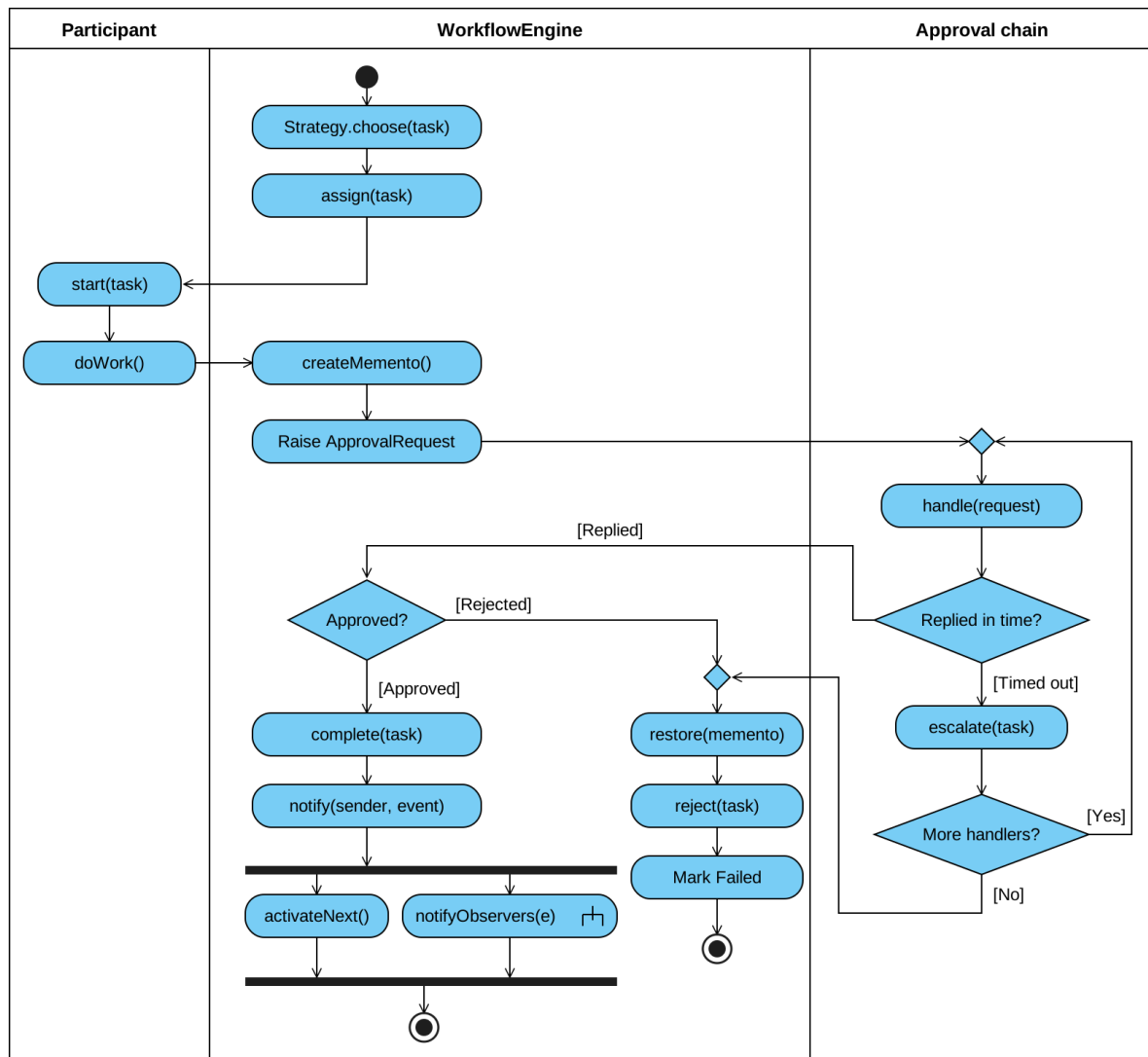
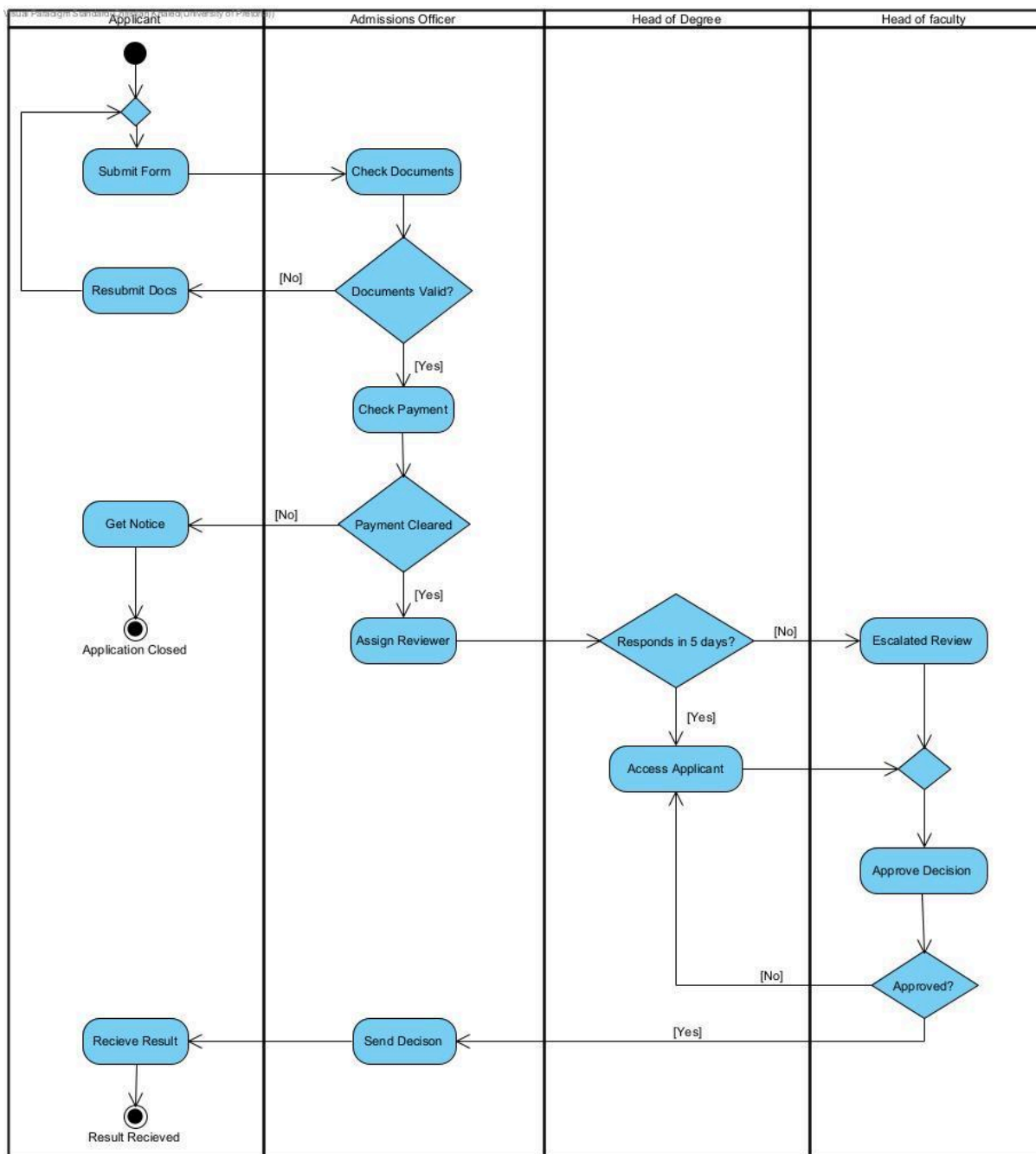


Diagram 3: Scenario based workflow diagram



Task 4:

- Builder: This creates the definitions stored in the facade
 - Builder: WorkflowBuilder
 - ConcreteBuilder: StandardWorkflowBuilder
 - Director: WorkflowDirector
 - Product: WorkflowDefinition

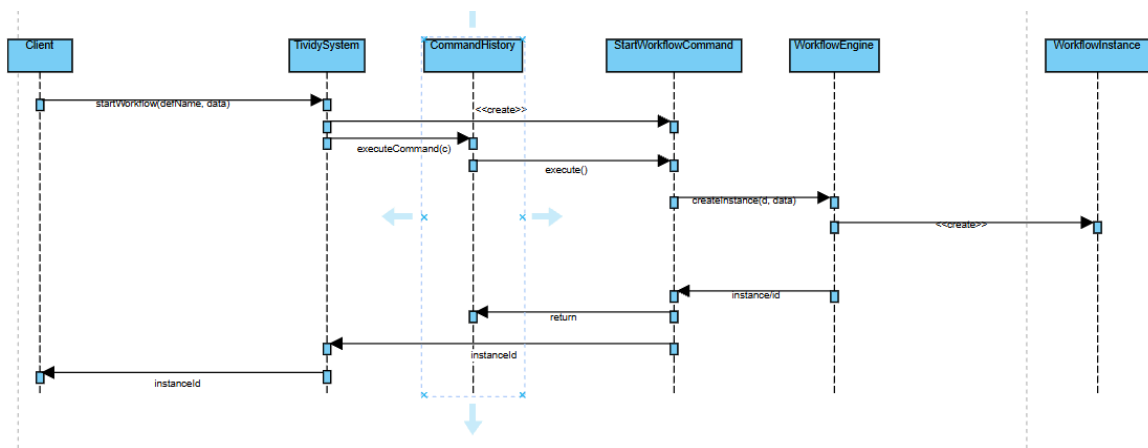
- Chain of Responsibility: This is the list of people tasks need to be approved from moving up it when it hits a certain time of waiting
 - Handler: ApprovalHandler
 - ConcreteHandler: RoleApprover, TimeOutEscalationHandler
 - Client: Task
- Command: This is used to send requests as objects to the engine (mediator). This means the request can for example start end or pause a task
 - Command: Command
 - ConcreteCommands: StartWorkFlowAssignment
 - Invoker: AssignTaskCommand, CompleteTaskCommand
 - Receiver: WorkflowEngine
 - Client: TividySystem
- Composite: This stores all our tasks in a tree structure to have them easily accessible for different stages of a large assignment. Eg, a student applies and the first stage is verify documents, this will have different tasks in it
 - Component: WorkNode:
 - Composite: Stage
 - Leaf: Task
 - Client: WorkflowEngine
- Decorator: This will wrap the task object with additional responsibilities, like a task could be wrapped in an international decorator so that there is extra functionality for example an international students (maybe they need their passports)
 - Component: WorkNode
 - ConcreteComponent: Task
 - Decorator: TaskDecorator
 - ConcreteDecorator: ValidationDecorator, DeadlineDecorator, PriorityDecorator

- Facade: This stores all the definitions of the TIVIDY system created from the builder. It takes a whole lot of actions to be done and brings them into one
 - Facade: TividySystem
 - Subsystem classes: WorkflowEngine, CommandHistory, WorkflowDefinition
- Iterator: This is used to iterate through the composite pattern.
 - Iterator: WorkNodeIterator
 - ConcreteIterator: DepthFirstIterator
 - Aggregate: WorkNode
 - ConcreteAggregate: Stage
- Mediator: This is the engine of the system that coordinates everyone and stores the current running instance doing whatever needs to be done to that instance
 - Mediator: WorkFlowMediator
 - ConcreteMediator: WorkFlowEngine
 - Colleague: WorkNode
 - ConcreteColleague: Task, Stage
- Memento: This will store the internal state of a task before the command pattern changes it. This ensures that if there is an error we can go back to the last save stored in the mementos history
 - Originator: Task
 - Memento: TaskMemento
 - Caretakers: AssignTaskCommand, CompleteTaskCommand
- Observer: This works with mediator, it notifies all the interested listeners in the system telling them what they need to know based of something changing
 - Subject: WorkflowSubject (Change that isnt in the UML yet, WorkFlowEngine inherits from both mediator and this subject)
 - ConcreteSubject: WorkflowEngine
 - Observer: WorkFlowObserver

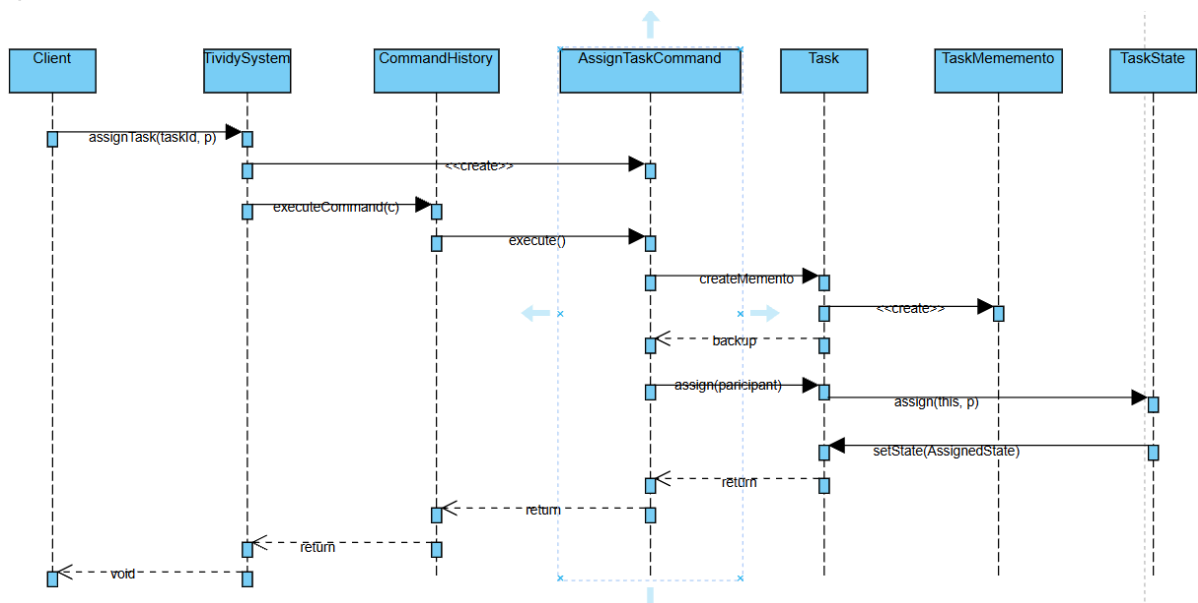
- ConcreteObservers: AuditLogged, NotificationsService, ProgressMonitor
- State: This stores the state of the actual task on where it is in the chain of responsibility. Is it approved, pending, escalated, critical, or denied
 - Context: Task
 - State: TaskState
 - ConcreteStates: AvailableState, AssignedState, inProgressState, CompletedState, RejectedState, EscalatedState
- Strategy: This will do the same thing in different ways and come up with the same outcome. Strategy in this TIVIDY system will assign a task in different ways.
 - Context: Task
 - Strategy: AssignmentStrategy
 - ConcreteStrategies: RoleBasedAssignment, LeastLoadedAssignment, RuleBasedAssignment
- Template Method: Every task in the composite tree has a template function that will ensure it can run
 - AbstractClass: WorkNode
 - ConcreteClasses: Task, Stage
- Prototype: Every definition has a task tree (composite) we use prototype to clone that tree for every instance.
 - Prototype: WorkNode
 - ConcretePrototypes: Task, Stage TaskDecorator
 - Client: WorkflowDefinition

Task 6: Sequence diagrams and State diagram

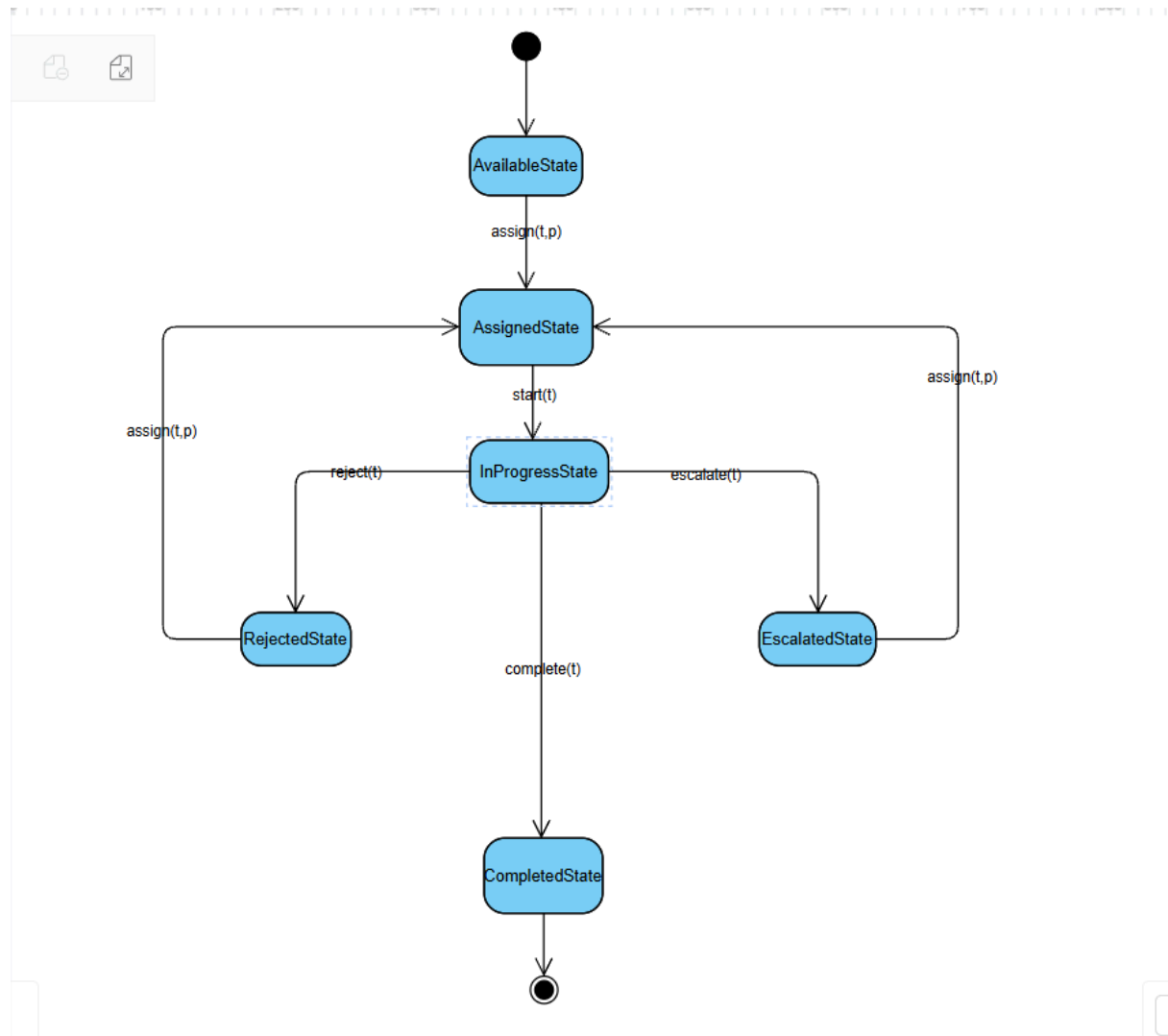
1)



2)



State:



Github Link:

https://github.com/u25420314/COS214_TheGithubGoliaths

